FluxMed

Otimização de rotas para medicamentos e insumos

Relatório técnico - versão 6

Tech Challenge - Fase 2 - Projeto 2

Álvaro Amorim de Oliveira Martins

Turma: 11IADT
Curso: Pós Tech - IA para Devs
Instituição: FIAP

2 de novembro de 2026

Código final e evidência experimental de 31/10/2026.
16 cenários sintéticos • 272 execuções únicas

Repositório: https://github.com/alvaro-amorim/fluxmed
Vídeo: https://youtu.be/gHc-irrihV0

VISÃO GERAL

Resumo e navegação

O algoritmo busca planos viáveis; atendimento e restrições precedem o custo.

O FluxMed é uma aplicação acadêmica de planejamento pré-despacho em Santa Aurora, uma cidade sintética. O GA evolui permutações de entregas; um decoder guloso forma rotas de múltiplos veículos. A avaliação considera prioridades, espera, disponibilidade de equipe e custo operacional com combustível. A solução final passa por validação independente antes de ser persistida.

Resultado principal. Na matriz de 16 cenários, GA + pruning + repair foi viável em **80/80** execuções, com média de **82,846 s**. GA + pruning foi viável em **62/80**. Nos 62 pares viáveis equivalentes, repair teve custo menor em 33, maior em 25 e igual em 4. São observações desta amostra, sem garantia de ótimo global.

Leitura	Páginas	Leitura	Páginas
Contexto, dados e arquitetura	3	Protocolo experimental	12
GA e referência FIAP	4	Resultados globais	13
Decoder e restrições	5	Escalas e cenários	14
Fitness e prioridades	6	62 pares comparáveis	15
Espera, equipe e custo	7	Caso oficial de 20 entregas	16
Exemplo: entradas e operadores	8	LLM e relatórios diários	17
Exemplo: rotas e custos	9	Reprodução e qualidade	18
Exemplo: avaliação e escolha	10	Limites e conclusão	19
Validação final e mapa	11	Referências e vídeo	20

Componentes da solução. Mapa interativo, comparação com heurísticas, relatórios operacionais PDF/XLSX/CSV por operação e assistente com grounding no backend. As páginas seguintes apresentam a representação genética, a formação das rotas e os critérios usados para comparar soluções.

01 / PROBLEMA

Contexto, dados e arquitetura

Planejar a distribuição antes da saída dos veículos, sem dados de pacientes.

O Projeto 2 pede representação genética para rotas, seleção, crossover, mutação e fitness que considere distância, prioridades e restrições, ampliando TSP para VRP. O FluxMed busca atender todas as entregas com uma frota heterogênea, recursos humanos compatíveis e retorno ao depósito dentro da jornada [1].

Entrada	Conteúdo utilizado
Entregas	ID, endereço, peso, volume, prioridade, tempo de serviço, janela HARD/SOFT, temperatura e manuseio.
Frota	Capacidade de peso/volume, autonomia, refrigeração, política de ajudante, propriedade/contrato e custos.
Equipe	Profissionais selecionados, papel, atividade, disponibilidade na data e regime de remuneração.
Cidade e operação	Grafo dirigido, depósito, trânsito/ocorrências, data e janela operacional.

Santa Aurora e operação-modelo

O benchmark usa o dataset sintético Santa Aurora 1.1.0 e a operação-modelo **OP-20260915-001**, de 15/09/2026, entre 07:30 e 18:00, com tráfego AUTOMATIC, sete veículos e 12 profissionais. Cada cenário gera suas próprias entregas em memória. O modelo de recursos é fixo; não representa uma operação hospitalar real.

Separação de responsabilidades

Camada	Responsabilidade
React / TypeScript / MapLibre	Planejamento, mapa, progresso via SSE e acesso ao assistente/relatórios.
FastAPI / aplicação	Autenticação, autorização, operação, atribuição de equipe, custo e persistência.
GA / routing	GA define a permutação; decoder atribui veículos; routing resolve trajetos no grafo dirigido.
PostgreSQL / PostGIS / LLM	Banco guarda fatos e geometrias; LLM consulta resultados autorizados e os explica.

Fonte: implementação local [2]; fixture acadêmico e manifesto do experimento [3].

02 / MOTOR GENÉTICO

Como o GA procura soluções

Cada gene é um ID de entrega. Ruas, veículos e profissionais não são genes.

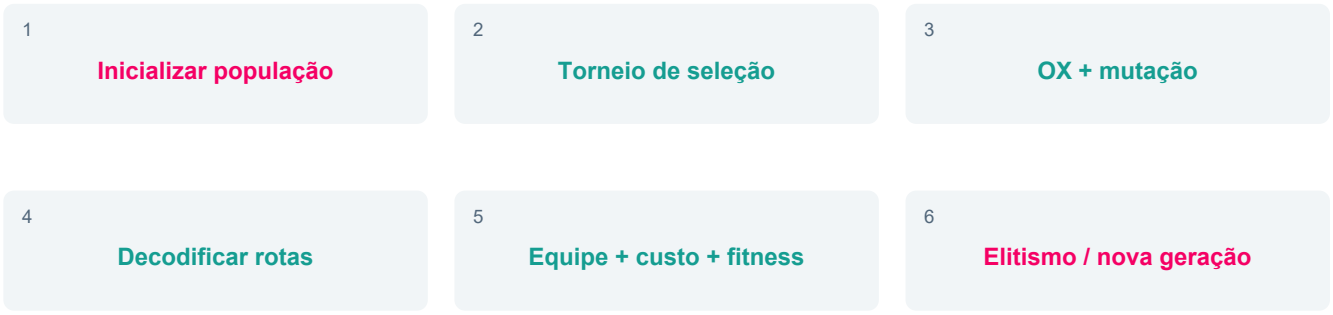


Figura 1. Ciclo evolutivo. A avaliação operacional ocorre em cada candidato, não apenas no plano final.

Componente	Comportamento implementado
Cromossomo	Permutação de todos os IDs, sem criação ou remoção de entregas.
População inicial	Nearest Neighbor, Best Insertion, variantes perturbadas e permutações aleatórias.
Seleção e cruzamento	Torneio (3 indivíduos no estudo) e Order Crossover (OX), preservando a permutação.
Mutação	Swap, insertion ou inversion; taxa adaptativa diante de estagnação/diversidade baixa.
Preservação e parada	Elitismo, imigrantes e término por gerações máximas ou estagnação.
Repair	Se há não atribuídas, intercala seus genes desde o início e reavalia; aceita somente se a fitness melhorar.

Referência conceitual da FIAP

O FluxMed tem implementação própria e usa o exemplo TSP da FIAP como referência conceitual e comparação técnica. No exemplo, uma permutação de cidades é avaliada por distância euclidiana e visualizada com Pygame. No FluxMed, a permutação contém IDs de entregas e é avaliada em uma matriz viária, com múltiplos veículos, restrições e fitness lexicográfica. A comparação fixa o commit **d236a2a** [4].

Correspondências: permutação como representação; seleção entre candidatos; OX preservando genes; mutação e elitismo. A tabela em docs/FIAP_TSP_BASE.md relaciona esses conceitos aos módulos atuais do FluxMed.

03 / DECODIFICAÇÃO

Do cromossomo às rotas

Uma sequência global é repartida em rotas por uma atribuição gulosa.

O decoder percorre os genes. Para cada entrega, examina os veículos e testa a rota prospectiva. Peso e volume são sempre limitados. Com advanced pruning, rejeita antecipadamente incompatibilidades de refrigeração/ajudante, autonomia, janela HARD e fim de jornada. Se não houver candidato, a entrega permanece não atribuída e a solução é inviável.

Escolha local de veículo

Entre as alternativas que passam pelos filtros ativos, minimiza-se esta tupla: **atraso SOFT ponderado prospectivo; risco CRITICAL; tempo marginal de viagem; distância marginal; peso restante; volume restante; ID do veículo**. Tempo e distância marginais incluem o efeito sobre o retorno ao depósito.

O custo operacional completo e as inversões globais de prioridade pertencem à fitness da solução formada. Não são os critérios locais de escolha do veículo. Logo, o GA explora diferentes permutações, mas o decoder não enumera todas as partições possíveis entre veículos.

Restrição	Aplicação
Peso / volume	Soma das entregas originais deve caber na capacidade de cada veículo.
Autonomia / retorno	Todos os trechos contam, incluindo depósito → primeira e última entrega → depósito.
Janela HARD	Início do atendimento depois do limite torna a solução inviável.
Janela SOFT	Atraso é permitido, com penalidade ponderada pela prioridade.
Temperatura / manuseio	Refrigeração e capacidade de ajudante compatíveis com as entregas.
Jornada / equipe	Retorno até o fim; equipe globalmente disponível e sem profissional reutilizado.

Ablação controlada. Desativar advanced pruning altera os filtros durante a construção das rotas. As restrições continuam contabilizadas na avaliação e na validação final, permitindo comparar o efeito das podas sob o mesmo critério de viabilidade.

04 / ORDEM REAL

Fitness: atendimento antes do custo

A comparação minimiza uma tupla, da esquerda para a direita.

Ordem	Campo	Regra
1	feasible	0 se viável; 1 se inviável.
2	hard_violations	Entregas não atribuídas, violações e falha de equipe.
3	service_penalty	Atraso SOFT ponderado pela prioridade.
4	critical_risk	Folga reduzida e atraso de entregas CRITICAL.
5	priority_inversion_minutes	Atendimentos de menor prioridade antes de maior prioridade.
6	total_cost_brl	Custo operacional com combustível e equipe efetiva.
7–10	makespan; viagem; distância; veículos	Desempates inferiores.

Quatro prioridades com uma regra explícita

LOW=1, NORMAL=2, HIGH=3, CRITICAL=4. Atraso é o tempo entre o início do atendimento e o fim da janela, quando positivo. Dez minutos de atraso SOFT contribuem com 10, 20, 30 ou 40, respectivamente.

Para CRITICAL, o risco soma **max(0; 30 − folga)**, quando existe fim de janela, e **3 × atraso**. A folga é fim da janela menos início do atendimento, em minutos. Uma CRITICAL atendida 10 minutos antes do limite contribui com 20; cinco minutos depois contribui com 35 + 15 = 50.

Para cada par de atendimentos em que o de menor prioridade começa antes do de maior, a inversão soma **(peso maior − peso menor) × intervalo entre inícios**. O cálculo abrange todas as rotas; prioridades iguais e inícios simultâneos não acrescentam penalidade.

Consequência: menor custo só desempata atendimento equivalente nos critérios anteriores. Não existe regra absoluta de atender toda CRITICAL primeiro, nem obrigação de mudar a rota a cada alteração de prioridade. É necessário haver alternativas comparáveis.

Fonte: FitnessVector.sort_key, FitnessEvaluator e RouteDecoder [2].

05 / COERÊNCIA DOS TOTAIS

Espera, equipe e custo operacional

O mesmo solver de equipe e o mesmo motor de custo são usados na busca e na persistência.

O relógio inclui a espera

Chegada = saída anterior + viagem. O início do serviço é o maior entre chegada e abertura da janela; a diferença é espera. Saída = início + serviço. A duração da rota é **viagem + serviço + espera**, incluindo retorno. Makespan é a maior duração, pois todas as rotas partem no início da operação.

Partida 08:00, viagem 10 min, abertura 09:00, serviço 5 min e retorno 10 min: chegada 08:10, espera 50 min, atendimento 09:00–09:05, retorno 09:15. A duração é **75 min**, que também entra nos componentes por hora.

Atribuição efetiva de equipe

OperationalFitnessEvaluator chama o TeamAssignmentSolver para matching determinístico de custo mínimo das rotas formadas. Considera papel, profissionais ativos/disponíveis selecionados, motorista ou courier, ajudante exigido e pessoal incluído no contrato. Um profissional não atende duas rotas. Sem equipe compatível, o candidato fica inviável **antes** da escolha final.

Componente	Cálculo no FinalCostEngine
Veículo próprio	km × depreciação/km + km × manutenção/km + horas × custo fixo/hora.
Agregado	Preço do contrato, conforme cadastro.
Combustível	Se não incluído: km / eficiência × R$ 6/L. É referência sintética, não cotação de mercado.
Equipe	Remuneração por hora, diária ou rota. Pessoal incluído no contrato não é cobrado novamente.

Espera aumenta os componentes por hora; contratos, diárias e valores por rota seguem seus regimes. Há cache limitado a 512 combinações por avaliador, sem compartilhar cache entre métodos/seeds. O mínimo do matching é condicionado às rotas, sem ótimo global de roteamento.

06 / EXEMPLO DIDÁTICO - 1 DE 3

Exemplo verificável: entradas e OX

Instância pequena, executada com as classes atuais; separada dos 272 resultados oficiais.

Considere depósito DEP e quatro entregas. Cada uma pesa 1 kg, ocupa 0,1 m³ e exige 5 minutos de serviço. Todo par de locais distintos tem **5 km e 10 min**; diagonal zero. A operação vai de 08:00 a 11:00. As janelas são HARD e terminam às 09:30; somente B abre às 08:30.

Entrega	Prioridade	Peso ordinal	Abertura
A	LOW	1	Sem limite inicial
B	CRITICAL	4	08:30
C	NORMAL	2	Sem limite inicial
D	HIGH	3	Sem limite inicial

V1 e V2 são próprios e idênticos: capacidade 2 kg e 1 m³, autonomia 30 km, refrigeração disponível, ajudante opcional e desnecessário. V1 tem o menor ID. Há dois motoristas ativos disponíveis, cada um a R$ 12/h; nenhum profissional está incluído em contrato. Combustível não incluído, eficiência 10 km/L.

Dois pais e um filho por Order Crossover

Pai 1: **[D, B, C, A]**. Pai 2: **[C, D, A, B]**. Fixam-se os índices 1 e 2 (base zero), herdando **[B, C]** nas posições centrais. Do pai 2, removem-se B e C; restam D e A. O OX preenche primeiro a posição 3 com D e depois a posição 0 com A:

```
Pai 1: D | B C | A
Pai 2: C D A B
Filho: A | B C | D
```

Uma mutação **swap** troca as posições 0 e 1 do filho e produz **[B, A, C, D]**. Insertion moveria um gene; inversion inverteria um segmento. O exemplo fixa os cortes e a troca para permitir conferência; na execução do GA essas escolhas usam o gerador pseudoaleatório controlado por seed.

Operadores chamados: order_crossover e swap_mutation. Os dois resultados preservam exatamente os quatro genes [2].

06 / EXEMPLO DIDÁTICO - 2 DE 3

Exemplo: formação de rotas e custo

Atribuição gulosa, cronologia e custo calculados pelo código atual.

No filho **[A, B, C, D]**, o decoder preenche V1 com A e B; a capacidade de 2 kg obriga C e D a V2. No mutante **[B, A, C, D]**, forma V1 com B e A; V2 continua com C e D. O tempo marginal favorece completar a rota existente, e os demais empates seguem a regra da página 5.

DEP → A → B → DEP
08:00 08:10 08:30 08:45

DEP → C → D → DEP
08:00 08:10 08:25 08:40

DEP → B → A → DEP
08:00 08:30 08:45 09:00

Figura 2. Horários de início do atendimento nos nós intermediários; último horário é o retorno ao depósito. A rota C → D é comum aos dois candidatos.

Rota	Viagem	Serviço	Espera	Duração	Custo
V1: A → B	30 min	10 min	5 min	45 min	R$ 42,00
V2: C → D	30 min	10 min	0 min	40 min	R$ 40,00
V1: B → A	30 min	10 min	20 min	60 min	R$ 48,00

Conferência monetária

Cada rota tem 15 km. Depreciação de R$ 0,60/km e manutenção de R$ 0,40/km somam **R$ 15**. Combustível: 15/10 × 6 = **R$ 9**. Custo fixo do veículo e motorista são R$ 12/h cada: **R$ 24/h**. Assim: custo da rota = 15 + 9 + 24 × duração/60.

A → B: 24 + 24 × 45/60 = **R$ 42**; C → D: 24 + 24 × 40/60 = **R$ 40**; B → A: 24 + 24 × 60/60 = **R$ 48**. Totais: **R$ 82** no filho e **R$ 88** no mutante. Não há atraso HARD, falta de equipe ou violação de autonomia/capacidade.

06 / EXEMPLO DIDÁTICO - 3 DE 3

Exemplo: fitness e escolha

Uma melhora de atendimento pode ser escolhida mesmo com custo maior.

Critério	Filho [A, B, C, D]	Mutante [B, A, C, D]
Viável / violações	Sim / 0	Sim / 0
Atraso ponderado / risco CRITICAL	0 / 0	0 / 0
Inversões de prioridade	150	60
Custo operacional	R$ 82,00	R$ 88,00
Makespan	45 min	60 min
Viagem / distância / veículos	60 min / 30 km / 2	60 min / 30 km / 2

Por que as inversões somam 150?

No filho, inícios: A e C às 08:10, D às 08:25 e B às 08:30. As contribuições são A→B: 3×20=60; A→D: 2×15=30; C→B: 2×20=40; C→D: 1×15=15; D→B: 1×5=5. Total: **150**. A e C simultâneos não geram inversão.

No mutante, A passa para 08:45 e B continua às 08:30. Restam C→B=40, C→D=15 e D→B=5: total **60**. A é LOW e ocorre depois das demais; não acrescenta inversão.

```
Filho: (0, 0, 0, 0, 150, 82, 45, 60, 30, 2)
Mutante: (0, 0, 0, 0, 60, 88, 60, 60, 30, 2)
```

O mutante vence: o primeiro critério diferente é a inversão, e 60 < 150. Os R$ 6 adicionais não revertem essa preferência, pois custo é comparado depois da qualidade. Isso demonstra o objetivo real do GA; não uma minimização isolada de reais ou quilômetros.

Mudança apenas de prioridade. Mantendo [A, B, C, D] e todas as demais entradas, trocar A de LOW para HIGH mantém as rotas e os R$ 82, mas muda a inversão de 150 para 80. Prioridade afeta a avaliação sem exigir uma alteração física de toda rota. Ambos os candidatos originais passaram pelo SolutionValidator.

07 / CONFIABILIDADE

Validação independente e mapa

A palavra “viável” do GA não é suficiente para persistir o plano.

SolutionValidator recebe as entregas, os veículos, a matriz de viagem e o início/fim da operação **originais**. Recalcula as restrições e compara o resultado com a solução decodificada; não confia apenas nos totais ou avisos já produzidos pelo decoder.

Conferência	O que é reconstruído
Integridade e uso de veículos	IDs conhecidos; todas as entregas uma única vez; nenhuma pendente; nenhum veículo em duas rotas.
Carga e volume	Soma das entradas originais e comparação com capacidades e totais declarados.
Percurso e autonomia	Todos os trechos, incluindo ida/retorno; distância, viagem, conexões e métricas não negativas.
Tempo e paradas	Quantidade e ordem; chegada, espera, início, saída, serviço e prioridade; janelas HARD e atraso declarado.
Retorno e jornada	Duração total com espera, retorno ao depósito e fim da operação.
Compatibilidade / equipe	Temperatura e política de ajudante no validador; atribuição global no solver de equipe.

Regressões cobertas

Os testes direcionados cobrem carga declarada abaixo da real, janela HARD violada com avisos do decoder removidos e reutilização do mesmo veículo em duas rotas, além de horários, retorno e custo operacional. A integração confronta o custo avaliado com o custo persistido, em banco dedicado.

Matriz e geometria usam a mesma referência

Cada par da matriz e sua geometria usam o horário inicial da operação como referência de partida. Dentro de cada caminho, o routing aplica fatores de trânsito pelo horário estimado de entrada nas arestas. Essa referência comum mantém a correspondência entre o percurso desenhado e as métricas. A acessibilidade viária é verificada na preparação da matriz.

Premissa temporal: o decoder reutiliza os tempos preparados por par. Serviço e espera alteram a cronologia dos atendimentos; os pares permanecem os mesmos durante a busca. O validador reconstrói o percurso a partir dessa matriz.

08 / EXPERIMENTO DE 31/10/2026

Protocolo experimental oficial

Entradas equivalentes dentro de cada cenário; configuração e seeds registradas.

Dimensão	Configuração
Perfis × escalas	NORMAL, BUSY, HIGH_PRESSURE e EMERGENCY × 20, 40, 60 e 80 entregas.
Geração de cenário	Uma seed por célula: 20260917. Total: 16 cenários.
Seeds de GA	101, 202, 303, 404 e 505 por célula e variante.
Contagem	32 runs determinísticos + 240 runs de GA = 272 execuções únicas.
População / gerações	48 indivíduos; até 120 gerações; torneio 3; crossover 0,90.
Diversidade e parada	Mutação 0,22 até 0,55; elitismo 0,05; estagnação 25; limiar de diversidade 0,35; imigrantes 0,10.

Método	Advanced pruning	Repair	Runs
Nearest Neighbor	Sim	Não	16
Best Insertion	Sim	Não	16
GA básico	Não	Não	80
GA + pruning	Sim	Não	80
GA + pruning + repair	Sim	Sim	80

Todos os métodos usam os mesmos recursos, entregas, matriz e janela de cada cenário, o mesmo avaliador operacional e a validação final independente. Mesmo no GA básico, restrições desativadas durante a construção continuam sendo violações; indisponibilidade global de equipe é avaliada em todos os métodos.

O tempo medido é o do método de otimização; exclui geração dos cenários e exportação. Cada método/seed recebe avaliador e cache próprios. A análise utiliza exclusivamente as 272 execuções da pasta oficial de 31/10/2026, identificadas por cenário, método e seed.

Fontes: matrix-manifest.json, experiment.json, benchmark-runs.csv e audit.json [3].

09 / VIABILIDADE E TEMPO

Resultados globais da matriz

Médias de tempo incluem todas as execuções, viáveis e inviáveis.

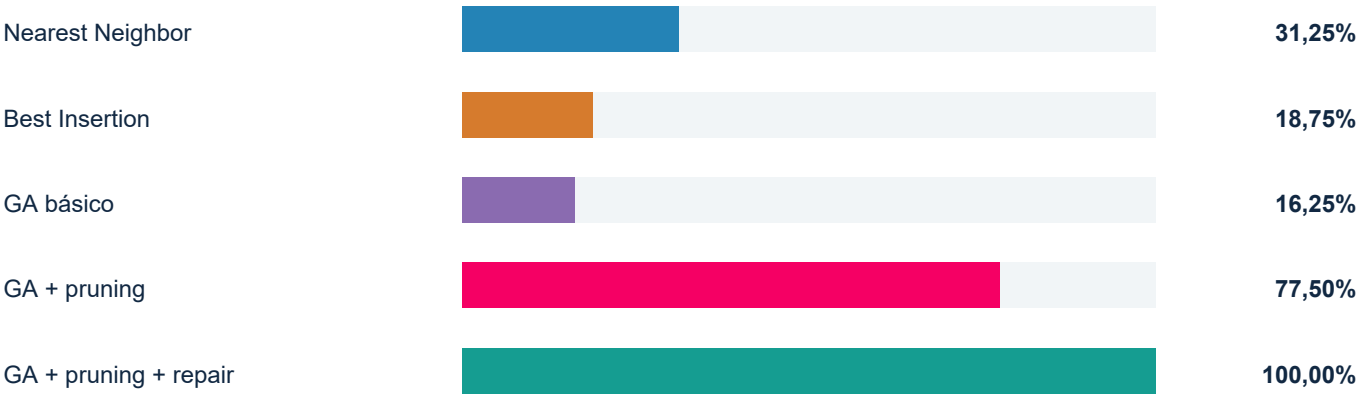


Figura 3. Viabilidade observada. Denominadores: 16 para heurísticas; 80 para cada GA.

Método	Viáveis	Taxa	Tempo médio
Nearest Neighbor	5/16	31,25%	0,015 s
Best Insertion	3/16	18,75%	0,111 s
GA básico	13/80	16,25%	51,325 s
GA + pruning	62/80	77,50%	49,810 s
GA + pruning + repair	80/80	100,00%	82,846 s

GA + pruning + repair obteve **18 execuções viáveis a mais** que GA + pruning (80 versus 62), com maior tempo médio. As heurísticas foram muito rápidas, mas frequentemente inviáveis. O GA básico teve 13/80 soluções viáveis: a busca evolutiva isolada, com esse decoder e orçamento, não eliminou os efeitos das restrições.

O resultado completo não prova que repair sempre reduz custo ou que toda instância terá solução viável. A análise econômica deve restringir-se a soluções viáveis equivalentes e considerar os critérios de atendimento que precedem custo.

Efeito observado das configurações. Com o mesmo orçamento de busca, GA básico, GA + pruning e GA + pruning + repair obtiveram, respectivamente, 13, 62 e 80 execuções viáveis. A comparação de custo e atendimento nos pares viáveis está na página 15.

10 / DISTRIBUIÇÃO DO RESULTADO

Escalas e cenários equivalentes

Viáveis/tentativas em cada célula; a mesma seed de cenário vale para todos os métodos.

Perfil	Entregas	NN	BI	GA	GA-P	GA-PR
NORMAL	20	1/1	1/1	5/5	5/5	5/5
NORMAL	40	1/1	0/1	5/5	5/5	5/5
NORMAL	60	0/1	0/1	0/5	5/5	5/5
NORMAL	80	0/1	0/1	0/5	5/5	5/5
BUSY	20	1/1	1/1	0/5	5/5	5/5
BUSY	40	0/1	0/1	0/5	5/5	5/5
BUSY	60	0/1	0/1	0/5	5/5	5/5
BUSY	80	0/1	0/1	0/5	3/5	5/5
HIGH_PRESS.	20	1/1	0/1	3/5	5/5	5/5
HIGH_PRESS.	40	0/1	0/1	0/5	5/5	5/5
HIGH_PRESS.	60	0/1	0/1	0/5	4/5	5/5
HIGH_PRESS.	80	0/1	0/1	0/5	0/5	5/5
EMERGENCY	20	1/1	1/1	0/5	5/5	5/5
EMERGENCY	40	0/1	0/1	0/5	5/5	5/5
EMERGENCY	60	0/1	0/1	0/5	0/5	5/5
EMERGENCY	80	0/1	0/1	0/5	0/5	5/5

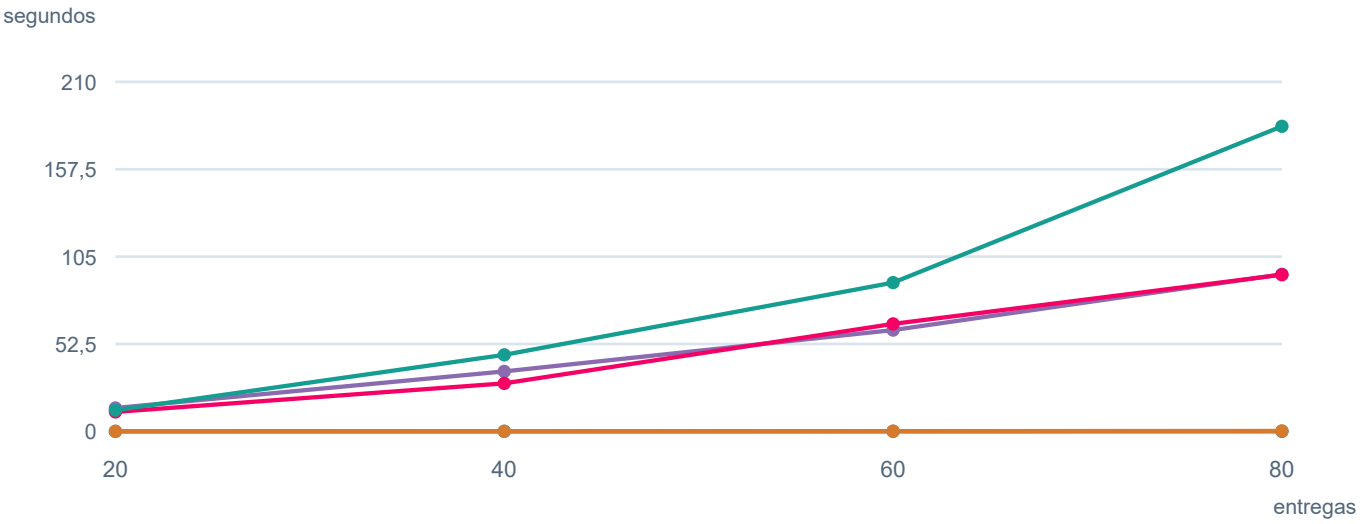


Figura 4. Tempo médio por escala, agregando os quatro perfis. Azul NN; laranja BI; lilás GA; rosa GA-P; verde GA-PR. Valores próximos de zero se sobrepõem.

As cinco seeds medem a aleatoriedade do GA sobre a mesma demanda de cada célula, não cinco cidades ou demandas independentes. A configuração completa foi viável em todas as células observadas; isso não constitui garantia fora da matriz.

11 / PRUNING × PRUNING + REPAIR

Os 62 pares viáveis comparáveis

Pareamento pelo mesmo cenário e pela mesma seed de GA, na matriz oficial.

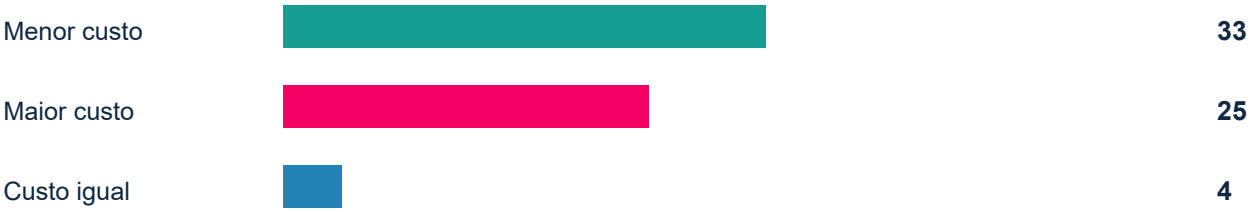


Figura 5. Contagem dos pares em que o custo operacional de repair é menor, maior ou igual ao de pruning.

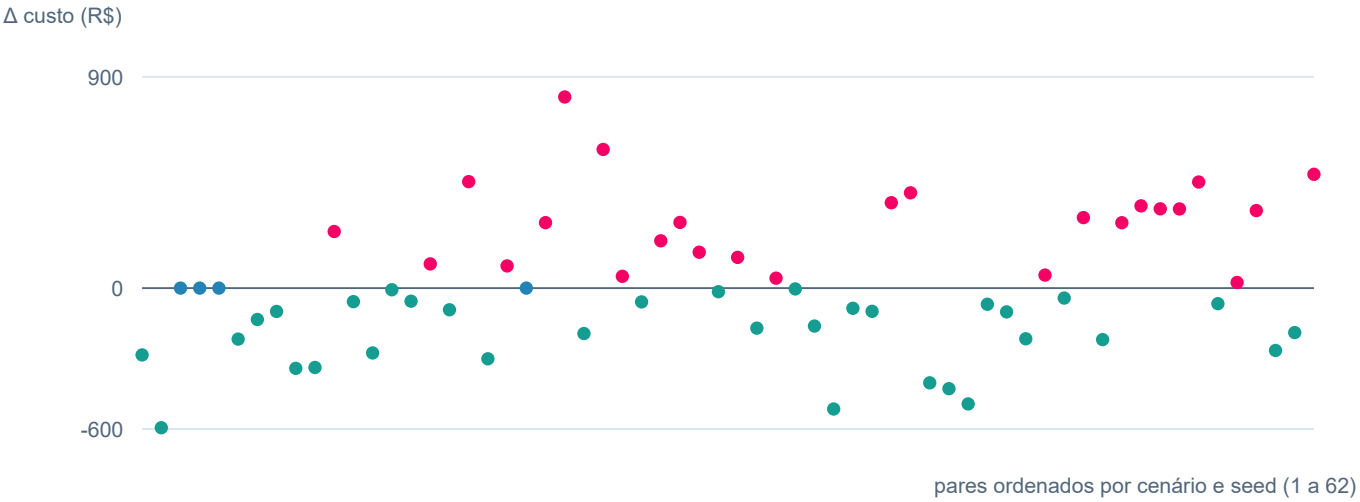


Figura 6. Δ = custo repair − custo pruning. Negativo favorece repair em custo; positivo favorece pruning. Ordem do CSV, não ordem temporal.

Comparação nos 62 pares	Melhor / menor	Pior / maior	Igual
Custo operacional de repair	33	25	4
Qualidade lexicográfica de repair	37	21	4

Qualidade compara, nesta ordem, atraso ponderado, risco CRITICAL e inversões. Assim, mesmo nos pares viáveis, um custo maior pode acompanhar atendimento melhor. Os grupos “menor custo” e “melhor qualidade” não são necessariamente os mesmos pares.

Repair aceita apenas melhora local de um candidato. Ainda assim, as variantes seguem trajetórias estocásticas distintas; a mesma seed não impõe as mesmas escolhas após caminhos de execução diferentes. Não se promete superioridade universal de custo, atendimento ou tempo.

Fonte: paired-pruning-repair.csv, 62 pares únicos; 18 casos inviáveis de pruning ficam fora deste pareamento [3].

12 / SOLUÇÃO DA MATRIZ OFICIAL

Caso oficial: NORMAL, 20 entregas

GA + pruning + repair; seed 101; escolhido como primeira seed, não como “melhor caso”.

Cenário **NORMAL-20-S20260917**: 20 entregas, 4 veículos, **190,290 km**, custo operacional de **R$ 1.575,27** e makespan de **544,93 min**. Sem entregas pendentes, sem violações HARD, atraso SOFT ponderado zero e risco CRITICAL zero. A inversão de prioridade é 1.282,319, portanto atendimento no prazo não significa inversão nula.

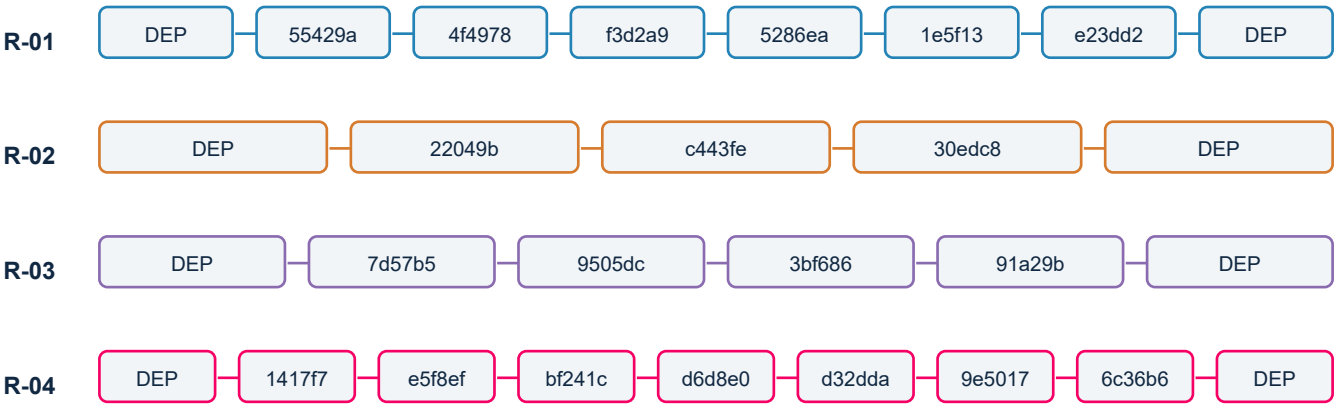


Figura 7. Sequências reais da solução oficial. Rótulos são os seis primeiros caracteres do ID de entrega; DEP é o depósito. Esquema de ordem, sem escala ou base cartográfica. No produto, a geometria é exibida no mapa de Santa Aurora.

Rota	Paradas	km	Viagem min	Serviço min	Espera min	Duração min
R-01	6	51,146	94,62	93	357,30	544,93
R-02	3	41,170	61,49	54	133,81	249,30
R-03	4	44,749	71,22	66	357,50	494,72
R-04	7	53,224	91,82	110	284,84	486,65

Cada duração fecha como viagem + serviço + espera; o makespan é o maior desses valores, não a soma. A primeira rota retorna aproximadamente às **16:34:56**, antes do fim às 18:00. Pequenas diferenças na soma de células decorrem do arredondamento de apresentação.

O run levou 18,335 s, completou 120 gerações e terminou por MAX_GENERATIONS. A melhor solução surgiu na geração 99. A validação independente registrada é válida e não contém ocorrências. O caso foi gerado em memória e preservado integralmente nos dados brutos do benchmark.

13 / INTERPRETAÇÃO FUNDAMENTADA

LLM, instruções e relatórios

A linguagem natural consulta fatos; o backend calcula as rotas e os custos.

O assistente recebe a pergunta e o contexto autorizado da operação, seleciona ferramentas de consulta e reúne dados estruturados de entregas, veículos, equipe, sequência, custos e métricas. Gemini ou mock redige a resposta com base nesse grounding. A LLM não participa do GA, não recalcula o fitness e não altera automaticamente o plano [5].

Uso	Tratamento
Pergunta factual	Consulta resultados persistidos e identifica suas fontes; valores devem coincidir com o plano.
Instruções ao motorista	Transforma paradas e restrições da rota em texto, preservando a sequência oficial.
Resumo diário	Interpreta a operação e os indicadores calculados pelo backend.
Sugestões	Propõe interpretações e melhorias em texto; não executa mudanças na operação.
Relatórios operacionais	Backend gera PDF, XLSX e CSV por operação, usando snapshot imutável do plano.

Engenharia de prompt e qualidade

As instruções delimitam o papel do modelo, exigem uso dos fatos disponíveis e reconhecimento de insuficiência de dados. A avaliação qualitativa documentada verifica pergunta factual, premissa falsa, dados insuficientes e instruções ao motorista. Mock permite testes gratuitos do fluxo; não comprova resposta de modelo pré-treinado.

A avaliação externa identifica operação e run, provider/modelo, pergunta, resposta e valores de referência. A conferência compara os fatos da resposta com as fontes operacionais. Os testes com mock verificam o fluxo com respostas controladas; a avaliação externa verifica a resposta efetivamente produzida pelo modelo.

Relatórios diários/semanais no enunciado. A implementação abrange o relatório diário por operação, com snapshot e exportação em PDF/XLSX/CSV. A consolidação semanal não foi implementada.

Credenciais permanecem no backend, fora do repositório, das telas de demonstração e deste relatório. Nenhuma chave é necessária para ler os artefatos experimentais.

14 / RASTREABILIDADE

Reprodução, versão e verificações

Resultados por cenário, fonte verificável e ambiente registrado.

A evidência oficial está em **benchmarks/results/final-evidence-operational-20261031/**. Contém CSVs de runs, resumos e convergência; relatório JSON com soluções e validação; manifesto; inputs/checkpoints por cenário; gráficos; auditoria e registro do ambiente. Há 272 chaves cenário/método/seed únicas e 19.144 pontos de convergência [3].

Fonte exata. experiment.json registra os hashes SHA-256 de 203 arquivos de código, testes, scripts, migrações e fixtures. source-snapshot.zip preserva esses arquivos e permite verificar a identidade da fonte medida. O conjunto exclui arquivos de ambiente, credenciais e banco de dados.

```
SHA-256 da fonte:
c4368d3b6e6ba81917b3ad47a6b28ad71ba73584a597ca616ab2a797c14a5830
```

Comandos na raiz, com o Python da API ativo

```
python scripts/seed_benchmark_template.py --seed

python scripts/benchmark_optimizer.py
--matrix-template-code OP-20260915-001 --preset final
--output benchmarks/results/minha-reexecucao
```

O primeiro comando requer uma instalação acadêmica recém-inicializada, com banco dedicado e catálogo demonstrativo. O segundo deve ser digitado em uma linha. Para retomar, repita-o com **--resume** e a mesma pasta explícita. Cada cenário concluído grava input/relatórios e só depois complete.json. Retomada confere fonte, parâmetros, ambiente, inputs e checksums; cenário incompleto pode ser repetido, sem duplicar os completos.

Validação e ambiente

O registro verification.json do estudo final documenta 28 testes direcionados aprovados, além de Ruff, MyPy e diff-check. Os testes cobrem custo operacional, equipe e persistência/retomada experimental; as regressões do algoritmo cobrem prioridades, espera e validação independente. Integrações usaram bancos dedicados. A identidade da fonte vincula as verificações ao código do experimento; contagens de subconjuntos de testes não são somadas como testes únicos.

Execução sequencial em Windows 11, Python 3.12.10, Intel Core i7-1355U (10 núcleos/12 processadores lógicos), PostgreSQL 16.4/PostGIS 3.4.3 em Docker dedicado. Tempos variam com hardware e carga.

15 / INTERPRETAÇÃO RESPONSÁVEL

Limites e conclusão técnica

Evidência de busca heurística sobre cenários sintéticos, com escopo explícito.

• Santa Aurora e as demandas são sintéticas. Há uma seed de demanda por célula; as cinco seeds de GA medem aleatoriedade da busca sobre a mesma entrada.

• 80/80 soluções viáveis caracteriza a configuração completa nas instâncias observadas. O resultado não garante viabilidade em qualquer demanda ou ótimo global.

• A fitness prioriza atendimento antes do custo. A comparação econômica exige soluções viáveis equivalentes e leitura conjunta dos critérios de qualidade.

• Os tempos refletem execução sequencial no ambiente registrado. Hardware, carga do sistema e tamanho das instâncias influenciam o custo computacional.

• Cada par da matriz parte da referência inicial da operação. O routing considera fatores temporais dentro do caminho; o decoder reutiliza esses tempos ao longo da rota.

• O decoder atribui veículos de forma gulosa; as permutações exploradas não enumeram todas as partições. O matching minimiza o custo de equipe para as rotas já formadas.

• As heurísticas têm uma execução por cenário e cada variante de GA tem cinco. As taxas possuem denominadores diferentes; a matriz caracteriza esta amostra controlada.

Conclusão

O FluxMed demonstra um GA de permutações com operadores próprios para roteamento, decoder de múltiplos veículos, avaliação de atendimento e custo operacional, validação independente e apresentação no mapa. Equipe e combustível compõem a avaliação operacional durante a busca e no plano final. A espera integra a cronologia, a duração e os custos por hora.

Na matriz oficial, pruning + repair obteve maior frequência de viabilidade que as outras configurações testadas, com maior tempo médio. O pareamento mostra benefícios e perdas de custo e qualidade, sem superioridade universal. O assistente transforma resultados calculados em instruções e explicações, respeitando seu papel de interpretação.

A validade empírica das conclusões está delimitada às instâncias sintéticas e ao protocolo experimental apresentados.

FONTES / ACESSO

Referências e links da entrega

Consulta técnica e documental: 2 de novembro de 2026.

[1] FIAP. Tech Challenge - Fase 2, Projeto 2.

Enunciado acadêmico, arquivo Tech Challenge.pdf, páginas 4–7: GA/VRP, restrições, mapa, LLM e entregáveis.

[2] FluxMed. Código-fonte e documentação.

github.com/alvaro-amorim/fluxmed. Fontes: optimization/{models,operators,engine,decoder,fitness,repair}.py; application/optimization/{evaluation,team_assignment,cost_engine,routing_matrix,service}.py. README e docs/OPTIMIZATION_SPEC.md descrevem a implementação.

[3] FluxMed. Evidência operacional final de 31/10/2026.

Diretório benchmarks/results/final-evidence-operational-20261031/: benchmark-runs.csv, benchmark-summaries.csv, benchmark-report.json, benchmark-convergence.csv, paired-pruning-repair.csv, matrix-manifest.json, experiment.json, source-snapshot.zip, audit.json, hardware.json e verification.json.

[4] FIAP. Genetic Algorithm TSP.

Commit d236a2ae6a08fefdb8ee9f1ca3f8201223a92da3, arquivos genetic_algorithm.py e tsp.py. Repositório oficial no commit analisado. Comparação técnica em docs/FIAP_TSP_BASE.md.

[5] FluxMed. Assistente e relatórios.

docs/LLM_IMPLEMENTATION.md, docs/ASSISTANT_EVALUATION.md e docs/REPORTS.md: providers, grounding, avaliação qualitativa, credenciais e escopo diário dos relatórios.

Identificação da versão

Relatório v6 baseado na fonte identificada em [3] e nas 272 execuções da matriz oficial de 31/10/2026. Os hashes por arquivo e o snapshot permitem conferir o código usado no experimento. O repositório [2] reúne a implementação e os guias técnicos da solução.

Vídeo de demonstração - até 15 minutos
Link do vídeo: https://youtu.be/gHc-irrihV0.